MIT WORLD PEACE UNIVERSITY

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory (CSE20110)

Laboratory Continuous Assessment 3

GitHub Link : Project_Github_link

CampusVault P2Ps

Peer-to-Peer File and Notes Sharing Platform

Submitted by

Sr. No.	Student Name	PRN
1	Aayush Patil	1262242184
2	Sushant Chaudhari	1262242201
3	Ruchita Chavan	1262242306
4	Abu Hadeed Siddique	1262242275

Panel: J Batch: J1

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr.Uma Pujeri

Academic Year 2026–2027

Submission Date: [22-10-2026]

CERTIFICATE

This is to certify that the mini-project entitled “CampusVault P2P: Peer-to-Peer File and Notes Sharing Platform” has been carried out by the following students of Third Year B.Tech. Computer Engineering and Technology, Panel [PANEL], Batch [BATCH], Group No. [GROUP NUMBER], under my guidance during the academic year 2026–2027.

Student Name	**PRN**
Aayush Patil	1262242184
Sushant Chaudhari	1262242201
Ruchita Chavan	1262242306
Abu Hadeed Siddique	1262242275

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature: _____

Dr. Uma Pujeri
Project Guide and Course Faculty

Date: _____ Place: Pune

Acknowledgement

We express our sincere gratitude to our project guide and Computer Networks Laboratory course faculty for their guidance and for providing the academic context in which this mini-project was developed. We also acknowledge the Department of Computer Engineering and Technology at MIT World Peace University for the laboratory learning environment and computing resources supporting this work.

We thank the Python and Flask documentation teams and the maintainers of the networking libraries used as references during implementation. The source code and project setup documentation are maintained in the CampusVault P2P GitHub repository. Any individual contribution or additional assistance should be acknowledged here accurately before submission.

Group No. [GROUP NUMBER]

Abstract

CampusVault P2P is a peer-to-peer (P2P) file and academic-notes sharing prototype developed for the Computer Networks Laboratory. It explores how peers can discover shared content through a tracker while transferring file payloads directly over TCP sockets, rather than routing all file data through a central file server. The project combines a Flask-based web interface, a tracker/peer registry, command-line peer nodes, chunked file transfer, and SHA-256 integrity verification. The repository describes a custom TCP framing protocol with a header, length-prefixed JSON metadata, and payload data; file content is transferred in 64 KB chunks. Each chunk is checked and a full-file hash is used as an integrity condition before a received file is accepted or reconstructed. The web application exposes note and upload-related endpoints, while the tracker maintains peer and file-hash metadata. This report documents the intended architecture, implementation modules, network protocol choices, reproducible startup workflow, and a test plan based on the project presentation and repository README. The supplied presentation marks execution results as pending validation, so this report does not claim that all test cases have passed or report unmeasured performance. The prototype demonstrates a practical learning platform for TCP communication, peer discovery, multithreading, message framing, and integrity checking; broader deployment would require further security, scalability, and cross-network work.

Keywords: Peer-to-Peer Networking, TCP Sockets, Peer Discovery, SHA-256, File Transfer

Table of Contents

1. Introduction and Project Objectives

1.1 Introduction and Background

1.2 Problem Statement

1.3 Scope

1.4 Objectives

2. Proposed Design and Requirements

2.1 Architecture and Network Topology

2.2 Tools and System Requirements

2.3 Addressing and Protocol Selection

3. Implementation and Technical Working

3.1 Methodology

3.2 Configuration or Code Implementation

3.3 Protocols and Technical Working

3.4 Execution Procedure

4. Testing and Results

4.1 Test Plan

4.2 Results and Execution Evidence

4.3 Objective Achievement

5. Discussion and Conclusion

5.1 Discussion and Comparative Analysis

5.2 Advantages and Applications

5.3 Limitations

5.4 Challenges and Troubleshooting

5.5 Future Scope

5.6 Conclusion

References

AI Usage Declaration

Appendices

Note: Page numbers should be updated in Microsoft Word after final editing. This manually prepared contents list is provided for navigation.

List of Figures

Figure 2.1 CampusVault P2P logical architecture

Figure 3.1 Suggested execution workflow

Figure 4.1 Placeholder for actual test execution evidence

List of Tables

Table 1.1 Project objectives and verification methods

Table 2.1 Tools and system requirements

Table 2.2 Network endpoints and roles

Table 4.1 Test plan and observed results

Table 4.2 Objective achievement

Table 5.1 Troubleshooting considerations

Table C.1 Individual contributions

1 Introduction and Project Objectives

1.1 Introduction and Background

File sharing is a common network application in academic environments, where students exchange notes, laboratory files, and other learning resources. In a conventional client–server design, clients contact a central server and the server commonly handles both discovery and delivery. This can simplify administration, but it can also make the central server a bottleneck and a single point of failure for file delivery.

A peer-to-peer (P2P) design allows participating nodes to provide content to other nodes. A tracker may coordinate discovery by keeping information about peers and the content they announce, while the actual file payload moves through direct peer connections. CampusVault follows this separation: the repository describes a tracker for peer discovery and TCP connections between peer nodes for file transfer. The project uses Python sockets and multithreading, a Flask web application, and SHA-256 hashing to check received content [1]–[4].

TCP provides a reliable, ordered byte stream, but it does not preserve application message boundaries. An application protocol must therefore define framing and know how much metadata or payload to read. The repository documents a custom frame using a P2P header, length-prefixed JSON metadata, and a payload [1], [3]. SHA-256 produces a digest that can be compared to detect whether content differs from the expected bytes; it is used here for integrity verification, not as encryption or proof of sender identity [4].

1.2 Problem Statement

The project addresses the need for a small academic file-sharing prototype that supports peer discovery and direct transfer while providing an integrity check on received files. A design that relies only on a central file server can concentrate transfer load at one point. A basic P2P transfer without a defined message protocol and integrity checks can also be difficult to validate. CampusVault combines tracker-based discovery, TCP transfer, chunk-level processing, and SHA-256 verification to demonstrate these networking concepts in one system.

1.3 Scope

The project scope includes a Flask web interface, server/tracker startup, peer registration and file-hash lookup, command-line peer nodes, direct TCP file transfer, chunked payload handling, and SHA-256 validation. The repository describes a local-network workflow and support for multiple machines on the same LAN. The current project presentation identifies the same-local-subnet requirement as a limitation. Public-internet deployment, production-grade authentication, encrypted transfer, access-control policy, and large-scale tracker operation are not claimed as completed features.

1.4 Objectives

Objective ID	Objective	Verification Method

O1	Provide a way to register or announce peers and discover shared files.	Run tracker/hub and check peer/file lookup using the UI or CLI.
O2	Transfer a selected file directly between peers over TCP.	Seed a sample file on one peer and fetch it from another; inspect transfer output.
O3	Check transferred data using SHA-256 at chunk and full-file levels.	Compare expected and received chunk/full-file digests in actual execution logs.
O4	Provide a reproducible web and command-line workflow.	Follow the repository setup instructions on a supported environment.

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

Figure 2.1 shows the logical division between the web/control path and the peer-to-peer data path. A browser communicates with the Flask web application over HTTP. The hub and tracker coordinate announcements and peer discovery. Once the requesting peer has the necessary peer address and port, file data is exchanged directly between peer nodes over TCP. The tracker is not intended to relay the file payload.

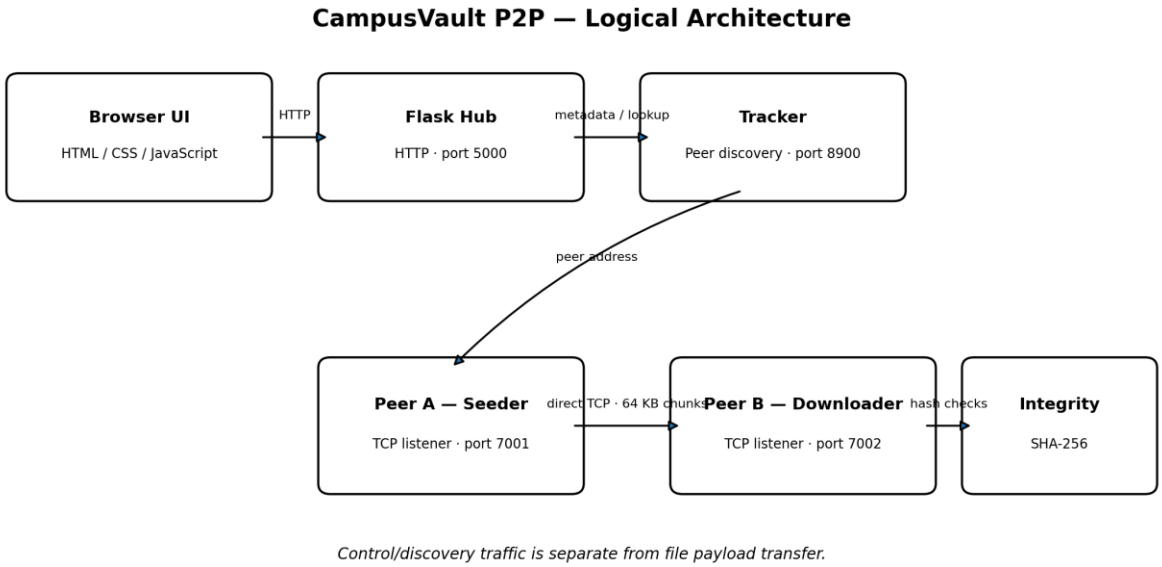


Figure 2.1 CampusVault P2P logical architecture

The repository identifies server.py as the Flask web server and tracker runner, peer_cli.py as the terminal peer utility, and the p2p_core package as the networking core. The protocol module handles framing,

chunking, and SHA-256; peer_node.py provides threaded TCP peer behavior; and tracker.py maintains peer registry information [1]. The web layer and tracker perform control and metadata functions, while direct TCP sessions carry file bytes.

2.2 Tools and System Requirements

Tool / Resource	Version or Specification	Purpose
Python	Python 3; exact tested version not specified in supplied materials	Application and peer-node implementation.
Flask	Dependency listed in requirements.txt; exact version not stated here	HTTP server and web UI endpoints.
Python socket	Standard library	TCP listener, connection, and byte transfer.
hashlib / SHA-256	Python standard library	Digest calculation and integrity comparison.
HTML, CSS, JavaScript	Browser-based frontend	Web dashboard and user interaction.
JSON	Metadata format	Peer/file metadata and protocol control messages.
Local network	Same LAN/subnet for documented multi-machine workflow	Reachability between peer nodes.
Git / GitHub	Repository: sushant-C7/CN-Project	Source-code hosting and setup instructions.

The exact operating-system versions, hardware specifications, dependency versions, and performance environment should be recorded from the machines used for the final demonstration. They are not inferred in this report.

2.3 Addressing and Protocol Selection

The project uses IP addresses and TCP ports to identify reachable services and peers. The README documents a web UI on port 5000, the hub seeding socket on port 7001, and the tracker on port 8900. A standalone CLI peer defaults to port 7002, and the peer-to-peer command examples specify the target peer IP and port. These ports are application configuration values, not reserved protocol ports.

Device / Service	Address / Port	Protocol	Role
Browser	127.0.0.1:5000 or hub LAN IP:5000	HTTP	Access web interface.
Flask hub / web server	0.0.0.0:5000 (as documented)	HTTP over TCP	Serve UI and API routes.
Tracker	0.0.0.0:8900 (as documented)	TCP socket / JSON framing	Register peers and support discovery.

Hub seeding peer	0.0.0.0:7001 (as documented)	TCP	Serve seeded files to requesting peers.
CLI downloader peer	Example port 7002	TCP	Request and receive file chunks.

The address 0.0.0.0 means a server binds to available local interfaces; it is not the address a remote peer should use as its destination. A peer should connect using the seeder machine's reachable LAN IP address and configured port. The README recommends checking host firewalls when a LAN connection is refused or times out [1].

TCP is selected for reliable, ordered transport. Since TCP is a byte stream, application-level framing is used to separate message headers, JSON metadata, and file payload. SHA-256 is selected as an integrity digest. It should not be interpreted as confidentiality, authentication, or a substitute for encrypted transport.

3 Implementation and Technical Working

3.1 Methodology

The intended implementation workflow, based on the repository documentation, is as follows:

1. Obtain the source code from the GitHub repository and install the dependencies listed in requirements.txt.
2. Start server.py to launch the Flask web interface, tracker, and hub seeding node.
3. Share or seed a file through the web interface or launch an independent seeder using peer_cli.py.
4. Register/announce the peer and make the file hash available to the tracker for discovery.
5. Start a requesting peer and supply the target peer IP, port, and file SHA-256 hash.
6. Establish the TCP connection, exchange protocol control messages, and transfer the file in chunks.
7. Validate each chunk and the complete file using SHA-256 before accepting the reconstructed file.

3.2 Configuration or Code Implementation

3.2.1 Flask web application

The repository lists server.py as the web server entry point. The presentation describes HTTP routes including /api/notes, /api/notes/upload, and /api/p2p/download. The web interface is stored under templates/ and static/ and presents the user-facing controls. Storage includes JSON note metadata and file payloads. The browser layer supports sharing and discovery workflows, while the socket peer modules handle direct transfer.

3.2.2 Tracker and peer registry

The tracker module maintains an in-memory peer registry and maps file hashes to peer identifiers, according to the project architecture in the presentation. This registry is used for discovery and coordination. The tracker supplies metadata needed for a peer to initiate a direct connection; it is not intended to forward the actual file bytes. Since the registry is described as in-memory, its state may not persist across process restarts unless persistence is implemented elsewhere.

3.2.3 Peer TCP module and message framing

The peer_node.py module implements the threaded TCP listener/client behavior described by the README. A listener can serve multiple connections using a thread per connection. The protocol description includes a P2P header, length-prefixed JSON metadata, and payload bytes. This framing helps the receiver determine how much control data and file data belongs to each protocol message.

The project presentation names the message sequence HANDSHAKE / HANDSHAKE_ACK, REQUEST_CHUNK, and CHUNK_PAYLOAD. It also describes 64 KB chunks. These labels describe the application-level protocol; they are not TCP flags or standardized Internet protocol messages.

3.2.4 Integrity verification

The sender and receiver use SHA-256 digests to check chunk content. The receiver compares the digest of each received chunk against the expected digest. A full-file SHA-256 comparison provides an additional check over the assembled content. The intended acceptance rule is to reconstruct/accept the file only when the relevant chunk checks and final file hash pass. Hash checking can detect accidental or unexpected data changes, but a secure system would also need authenticated metadata to defend against an attacker who can replace both content and expected hashes.

3.3 Protocols and Technical Working

The overall operation has two logical paths. The control path carries HTTP requests between the browser and Flask and metadata/announcements between peers and the tracker. The data path carries file payloads over a direct TCP connection between the serving peer and the requesting peer. Keeping these paths separate reduces the need for the tracker to relay the payload.

Stage	Message / Action	Purpose
1	Peer announcement / registration	Make peer identity and available file hashes discoverable.
2	Tracker lookup	Find a peer associated with the requested file hash.
3	TCP connection and HANDSHAKE	Initiate a transfer session and establish protocol context.
4	HANDSHAKE_ACK	Confirm the peer's response to the handshake.

5	REQUEST_CHUNK	Request a file chunk from the serving peer.
6	CHUNK_PAYLOAD	Send chunk metadata and payload to the receiver.
7	SHA-256 checks	Validate each chunk and the complete file before acceptance.

3.4 Execution Procedure

The following commands are taken from the repository's documented workflow. Run them from a terminal in an environment where Python and the listed dependencies are installed.

Step 1: Obtain the repository

```
git clone https://github.com/sushant-C7/CN-Project.git

cd CN-Project

pip install -r requirements.txt
```

Step 2: Start the hub server

```
python server.py
```

Open http://127.0.0.1:5000 in a browser on the same machine. For LAN access, use the LAN IP printed by the application and port 5000, subject to firewall and network configuration.

Step 3: Seed a file

Use the web interface's file-sharing/seeding action, or run a CLI seeder. Example command from the README:

```
python peer_cli.py --node "Seeder-PC" --port 7001 --seed-file "study_guide.pdf"
```

Step 4: Fetch from a second peer

Use the actual file hash printed by the seeder and the reachable LAN IP address of the seeder machine. Example command structure:

```
python peer_cli.py --node "Laptop-Peer" --port 7002 --fetch-hash "<SHA-256-HASH>" --peer-ip
<SEEDER-LAN-IP> --peer-port 7001
```

Replace all example names, file paths, hashes, and IP addresses with values from the actual test environment. Do not use the literal placeholder hash or an unreachable address.

Step 5: Verify

Record the transfer output, received filename and size, chunk verification messages, and full-file SHA-256 result. Capture screenshots or terminal logs for the final report. The supplied presentation does not include actual evidence images and labels the demonstration outcomes as pending validation; therefore, the procedure above is reproducible guidance, not proof that a run has succeeded.

Figure 3.1 Execution workflow: start server → start peers → discover file → transfer chunks → verify hashes.

4 Testing and Results

4.1 Test Plan

Testing should be carried out on the actual implementation and the results recorded from terminal output, web UI behavior, and received files. The test plan below defines expected behavior and evidence to collect. The observed-result column is deliberately marked pending because the supplied slide deck states that validation evidence has not yet been entered.

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	Start server.py with dependencies installed.	Flask UI and tracker start and listen on configured ports.	Pending validation; record startup output.	Not yet recorded
T2	Seed a sample text/PDF file using the UI or CLI.	Seeder loads file and exposes its hash for discovery.	Pending validation; record filename, size, hash.	Not yet recorded
T3	Register two peers and query a known file hash.	Tracker returns an appropriate peer/file match.	Pending validation; capture lookup result.	Not yet recorded
T4	Fetch a seeded file from another peer over TCP.	Receiver obtains the selected file in chunks.	Pending validation; capture transfer log.	Not yet recorded
T5	Compare chunk and final SHA-256 checks.	Valid transfer passes digest checks; mismatched content is rejected.	Pending validation; record actual digest output.	Not yet recorded
T6	Attempt a connection with incorrect IP/port or blocked firewall.	Connection fails or times out with diagnosable output.	Pending validation; record exact error and recovery.	Not yet recorded

4.2 Results and Execution Evidence

The presentation's validation dashboard explicitly lists server startup, file transfer, peer discovery, and integrity verification as pending validation. No actual terminal screenshot, packet capture, measured transfer time, throughput, or file-size result was supplied in the presentation. Consequently, this section records the current evidence status rather than reporting unobserved outcomes.

Evidence item	What to capture	Current status
Server startup	Terminal output showing Flask and tracker startup and bound ports.	Not supplied; add actual screenshot/log.
Peer discovery	Registered peer/file hash and tracker lookup output.	Not supplied; add actual output.

File transfer	Sender and receiver logs, file name and size, and completion message.	Not supplied; add actual output.
Integrity check	Per-chunk validation and final SHA-256 comparison result.	Not supplied; add actual output.
End-to-end demonstration	Screenshot of UI plus receiving peer output.	Not supplied; add actual screenshot.

[INSERT ACTUAL SCREENSHOT OF SUCCESSFUL TEST RUN HERE]

Figure 4.1 Test execution evidence (to be replaced with a genuine screenshot from the implementation).

4.3 Objective Achievement

Objective ID	Evidence / Test ID	Outcome
O1	T3 — peer discovery	Implementation intent documented; empirical result pending.
O2	T4 — direct TCP transfer	Implementation intent documented; empirical result pending.
O3	T5 — SHA-256 verification	Integrity-check mechanism documented; empirical result pending.
O4	Execution procedure in Section 3.4	Commands documented from repository; environment-specific execution should be recorded.

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

CampusVault illustrates a hybrid P2P arrangement: peer discovery is coordinated by a tracker, but file payloads are transferred directly between peers. Compared with a central file-server approach, this architecture can reduce the requirement for a central component to carry every file byte. It also creates a dependency on the availability and reachability of the selected seeder. The tracker remains a coordination dependency for discovery in the current design.

The project also demonstrates an important distinction between transport reliability and application integrity. TCP provides reliable ordered byte delivery during a connection, but the application still benefits from validating that the received file matches the expected content digest. Chunk-level checks

can identify a corrupted chunk, while a full-file digest checks the assembled file. No quantitative performance comparison is provided because the supplied materials contain no measured throughput, latency, concurrency, or file-size test results.

5.2 Advantages and Applications

- Direct peer-to-peer file payload transfer after discovery, rather than routing all payload bytes through the tracker.
- Use of TCP sockets to demonstrate connection establishment and reliable stream communication.
- Chunked transfer using the 64 KB chunk size described in the project materials.
- SHA-256 digest checks at chunk and full-file levels to detect content mismatches.
- Both browser-based interaction and a command-line peer workflow.
- Potential use in controlled classroom or laboratory environments for exchanging notes, sample datasets, and learning materials.

5.3 Limitations

- The presentation identifies same-local-subnet operation as a current limitation; different networks may require additional networking configuration or a relay/tunnelling solution.
- The tracker registry is described as in-memory, so persistence and recovery behavior require further evaluation.
- The supplied materials do not establish production-grade authentication, authorization, encryption, or protection against malicious peers.
- A SHA-256 match verifies content equality against the expected digest but does not prove the sender's identity if the digest itself can be replaced by an attacker.
- Scalability, concurrent transfer behavior, large-file handling, and throughput have not been quantified in the supplied test evidence.
- Observed test results and demonstration screenshots remain to be added.

5.4 Challenges and Troubleshooting

The repository README documents likely connection problems and recommended checks. The table below records these as troubleshooting guidance, not as claims that the team encountered each issue during its implementation.

Issue / Symptom	Likely cause / diagnosis	Suggested solution and verification
Connection refused or timeout	Target peer is not listening, wrong IP/port, firewall block, or machines cannot reach each other.	Confirm peer is running, verify LAN IP and port, allow Python through the private-network firewall, retry and record output.

Different Wi-Fi networks	Peers may not have direct LAN reachability.	Use the same local subnet for the documented baseline; treat cross-network connectivity as future scope.
File not discoverable	Peer/file announcement missing, wrong hash, or tracker state not populated.	Confirm seeding output and exact SHA-256 hash, then repeat lookup.
Integrity check mismatch	Payload differs from expected chunk/full-file digest or framing/reassembly issue.	Do not accept the file; capture logs, compare hashes, and repeat with a known sample file.
Port already in use	Another process is bound to the configured port.	Identify the process or choose an unused configured port consistently on sender and receiver.

5.5 Future Scope

- Add encrypted peer-to-peer sessions using an established secure transport and appropriate certificate/key management.
- Introduce authentication and authorization to control who can announce, download, or seed academic files.
- Persist tracker metadata and add stale-peer expiry, health checks, and restart recovery.
- Support discovery and transfer across different networks with carefully designed NAT traversal or relay support.
- Improve resilience with retry logic, timeouts, resumable transfers, and explicit handling of incomplete chunks.
- Benchmark throughput and latency across file sizes, peer counts, and LAN conditions, documenting the test environment.
- Add automated unit and integration tests for protocol framing, chunk ordering, corrupted payloads, peer disconnects, and malformed messages.

5.6 Conclusion

CampusVault P2P is a Python-based academic prototype for peer discovery and direct file transfer over TCP. Its documented architecture combines a Flask web interface, a tracker/peer registry, command-line peer nodes, custom message framing, chunked transfer, multithreading, and SHA-256 integrity checks. The project provides a useful practical setting for studying application-layer protocols, TCP socket programming, separation of control and data traffic, and file-integrity validation. The repository documents a reproducible startup and peer-transfer workflow, but the supplied presentation marks core execution tests as pending. The final submission should therefore include genuine test logs, screenshots, and measured observations before claiming successful validation. Encryption, access control, tracker

persistence, cross-network connectivity, and scalability testing remain logical areas for future development.

References

[1] Sushant-C7, “CN-Project: CampusVault P2P — Computer Networks Project,” GitHub repository. [Online]. Available: https://github.com/sushant-C7/CN-Project. Accessed: Oct. 22, 2026.

[2] Pallets Projects, “Flask Documentation.” [Online]. Available: https://flask.palletsprojects.com/. Accessed: Oct. 22, 2026.

[3] Python Software Foundation, “socket — Low-level networking interface,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/socket.html. Accessed: Oct. 22, 2026.

[4] Python Software Foundation, “hashlib — Secure hashes and message digests,” Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/hashlib.html. Accessed: Oct. 22, 2026.

[5] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/. Accessed: Oct. 22, 2026.

AI Usage Declaration

AI assistance was used to help organize and draft report prose from the supplied project template, presentation, and publicly accessible GitHub repository documentation. The team must review the final document against its actual source code and execution, verify all technical statements, replace placeholders, and insert genuine test evidence. No AI-generated test results are presented as experimentally verified.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	**PRN**	**Signature**
Aayush Patil	1262242184	_____
Sushant Chaudhari	1262242201	_____
Ruchita Chavan	1262242306	_____
Abu Hadeed Siddique	1262242275	_____

Appendices

Appendix A — Code and Configuration

The complete source code is maintained in the project repository. The principal files identified in the repository are listed below. Include selected code excerpts or full listings only where required by the course submission rules; remove credentials, secrets, and machine-specific private information.

File / Directory	Responsibility
server.py	Flask web server and hub/tracker startup.
peer_cli.py	Command-line peer node for seeding and fetching.
p2p_core/protocol.py	Protocol framing, chunking, and SHA-256 utilities.
p2p_core/peer_node.py	TCP peer listener/client and transfer handling.
p2p_core/tracker.py	Peer registry and file-hash discovery.
templates/index.html	Web dashboard HTML.
static/css/styles.css	Web interface styling.
static/js/app.js	Browser-side interaction logic.
requirements.txt	Python package dependencies.
storage/notes_data.json	Note/file metadata storage described by the project.
storage/files/	Stored file payloads described by the project.

Appendix B — Additional Evidence

Attach genuine evidence from the final execution here. Recommended items:

- Screenshot B.1: Flask server and tracker startup output.
- Screenshot B.2: Seeder announcing a sample file and displaying its SHA-256 hash.
- Screenshot B.3: Tracker peer/file lookup output.
- Screenshot B.4: Downloader connection, chunk transfer, and completion output.
- Screenshot B.5: Chunk-level and full-file SHA-256 verification output.
- Test log: operating system, Python version, dependency versions, peer IPs/ports, sample file name and size, observed transfer time if measured, and pass/fail result.

Appendix C — Individual Contributions

Replace the rows below with the actual contributions agreed by the project group. Do not assign work to a member without confirming that it was performed.

Student Name and PRN	Work Completed	Evidence / Deliverable
Aayush Patil	[Architecture / tracker / other actual contribution]	[Files, commits, or test evidence]
Sushant Chaudhari	[Peer protocol / transfer / other actual contribution]	[Files, commits, or test evidence]
Ruchita Chavan	[Web UI / integration / other actual contribution]	[Files, commits, or test evidence]
Abu Hadeed Siddique	[Testing / documentation / other actual contribution]	[Files, commits, or test evidence]

Appendix D — Supplied Implementation Screenshots

The following figures were supplied as project evidence. They document the visible web interface and seeder-side setup. They should be retained alongside the main testing section; they do not by themselves establish that a second peer successfully downloaded and reconstructed the file.

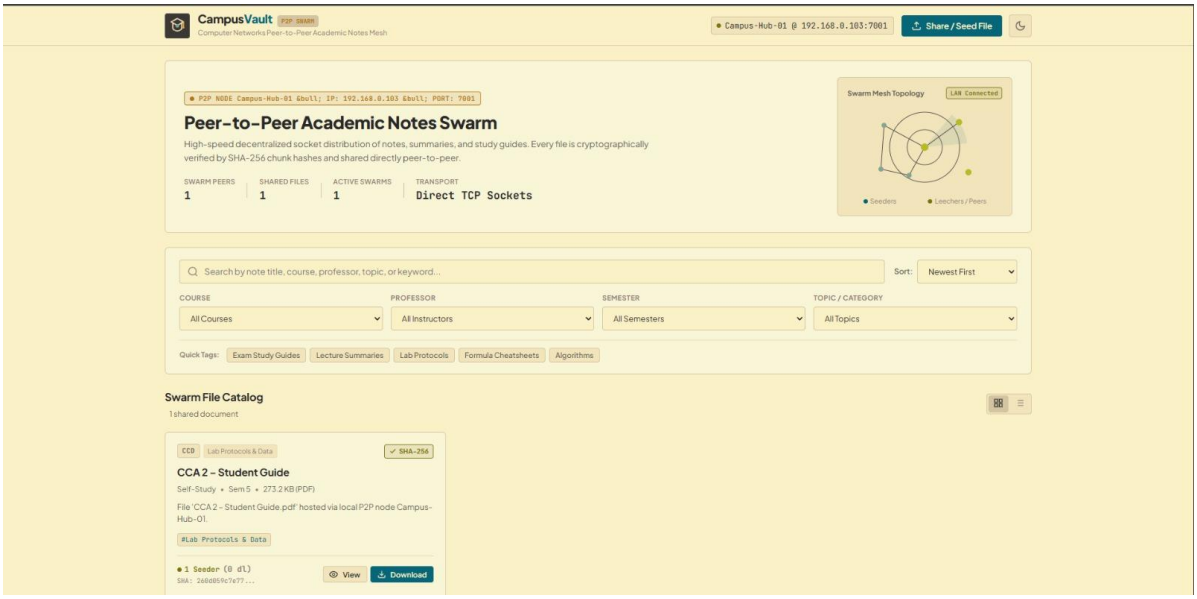


Figure D.1 — CampusVault P2P dashboard and shared-file catalog.

The dashboard shows the CampusVault interface, a local hub peer identified as Campus-Hub-01 at 192.168.0.103:7001, direct TCP sockets as the transport label, and one listed document ("CCA 2 – Student Guide"). The visible catalog displays a SHA-256 badge.

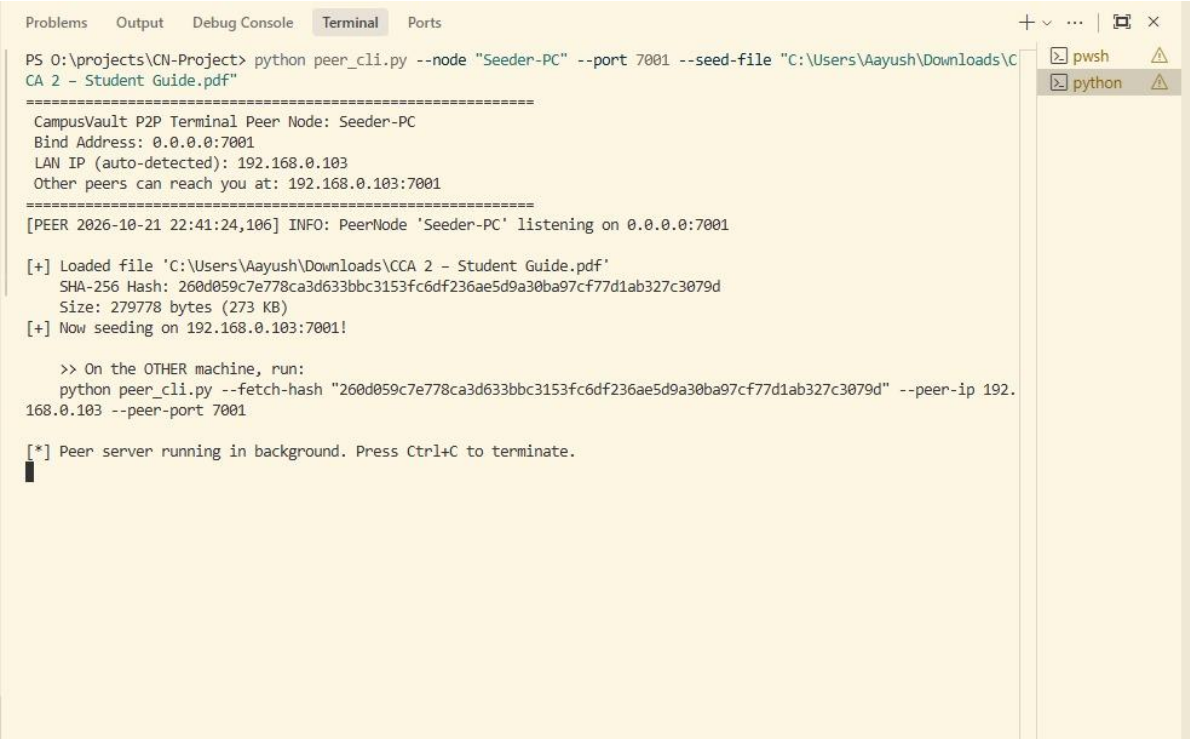


Figure D.2 — Seeder peer running in the terminal.

The terminal output shows the peer CLI launched with a seed file, listening on 0.0.0.0:7001, and advertising LAN address 192.168.0.103:7001. It displays a SHA-256 hash and file size of 279,778 bytes (273 KB), then reports that it is seeding. This is evidence of seeder startup and file loading.

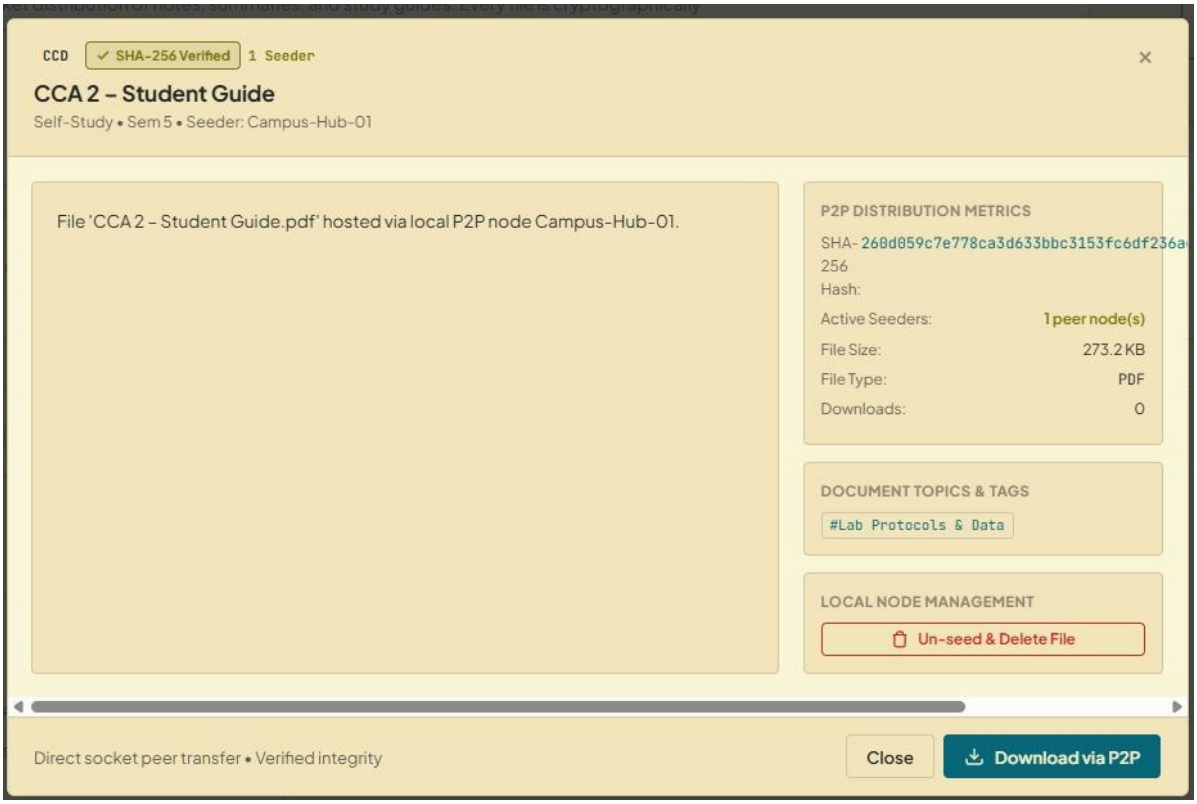


Figure D.3 — Shared document details and integrity indicator.

The document details view displays “SHA-256 Verified,” one active seeder, PDF type, and a file size of 273.2 KB. This demonstrates the UI’s displayed metadata and verification indicator; it is not independent proof of a completed download by another peer.

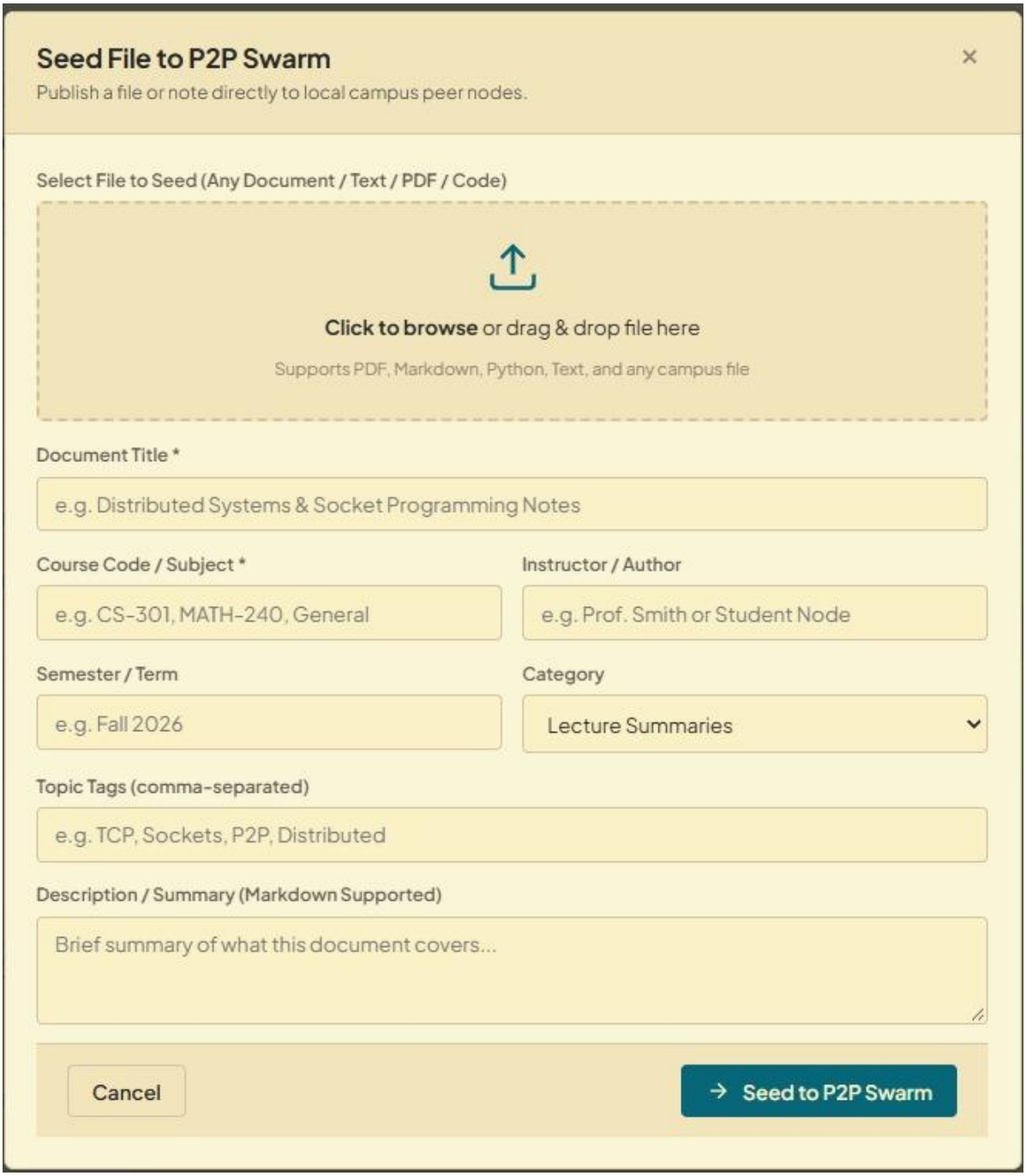


Figure D.4 — Share/Seed File form.

The form provides fields for the file, document title, course code/subject, instructor/author, semester/term, category, tags, and description. It documents the metadata available when publishing a file to the P2P swarm.